

DBMS

Project Report: Workforce Management System – Attendance, Shift, and Leave Tracking

Abstract

The Workforce Management System is a comprehensive database-driven application designed to automate and streamline core Human Resources (HR) operations. This project transitions traditional, manual employee tracking methods into a centralized digital platform. The core database schema handles employee records, shift scheduling, real-time attendance logging, and leave request workflows. To deliver a seamless user experience, the system's front end and middleware are developed using a hybrid stack of Node.js and Python, ensuring efficient API routing, real-time data processing, and an intuitive user interface for both administrators and employees.

1. Introduction

1.1 Project Overview

Managing workforce logistics in a growing organization requires strict data integrity and efficient business logic. This project provides a robust relational database backend to track employees across various departments and shifts while logging daily attendance and managing leave approvals.

1.2 Problem Statement

Manual HR tracking systems are prone to human error, data redundancy, and inefficiencies in payroll calculation. This project replaces manual tracking to streamline employee attendance, shifts, and leaves, strictly enforcing data integrity and organizational business logic (such as preventing overlapping leaves and accurately logging active shift hours).

2. System Architecture & Technology Stack

The project adopts a multi-tier architecture to separate data storage, business logic, and user interaction:

- **Database Layer (Backend):** Relational Database Management System (RDBMS) using SQL for structured data storage, DDL (Data Definition Language) for schema creation, and DML (Data Manipulation Language) for data ingestion.
- **Application & API Layer (Node.js):** Node.js serves as the primary backend environment, handling asynchronous database connections, executing SQL queries, and serving RESTful APIs to the front end.
- **Frontend & Analytics Interface (Python):** Python is utilized to build the front-end interfaces and dashboard analytics (utilizing frameworks such as Streamlit, Flask, or

Dash). It connects to the Node.js APIs to visualize attendance trends, render employee tables, and process data insights for management.

3. Methodology & Database Design

The core methodology relies on a normalized relational database design to eliminate redundancy. The system is built upon five primary entities:

3.1 Entity-Relationship (ER) Mapping

- **DEPARTMENT:** Dept_ID (PK), Dept_Name (Unique), Manager_ID.
- **SHIFT:** Shift_ID (PK), Shift_Name (Unique), Start_Time, End_Time.
- **EMPLOYEE:** Emp_ID (PK), Name, Email (Unique), Phone (Unique), Hire_Date, Dept_ID (FK), Shift_ID (FK).
- **ATTENDANCE:** Log_ID (PK), Emp_ID (FK), Date, Clock_In, Clock_Out (NULL allowed for active shifts), Status.
- **LEAVE_REQUEST:** Leave_ID (PK), Emp_ID (FK), Leave_Type, Start_Date, End_Date, Approval_Status.

Relationship Structure:

- **1-to-Many (1:M):** One Department employs many Employees.
- **1-to-Many (1:M):** One Shift is assigned to many Employees.
- **1-to-Many (1:M):** One Employee has multiple Attendance logs.
- **1-to-Many (1:M):** One Employee submits multiple Leave Requests.

3.2 Constraint Management & Circular Dependencies

A key design decision involved the Manager_ID attribute in the DEPARTMENT table. Because a Manager is also an Employee, linking them creates a circular dependency during table initialization. This was resolved by creating the tables sequentially and enforcing the foreign key constraint post-creation using an ALTER TABLE execution.

4. Implementation & Data Analytics

4.1 Data Ingestion (DML)

The database was populated with a robust sample size of 30 distinct employee records spanning 7 departments and 5 shift types. To rigorously test the system's analytical capabilities, varying edge cases were injected, including 30 attendance logs (capturing late arrivals, half-days, active shifts with NULL clock-outs, and overtime) and 15 leave requests with varying approval statuses.

4.2 Operational Querying

The Node.js and Python interfaces utilize specific SQL queries to render dashboards. Key operational queries include:

Extracting Approved Sick Leaves:

SQL

```
SELECT e.Emp_ID, e.Name, l.Leave_Type, l.Start_Date, l.End_Date, l.Approval_Status
FROM LEAVE_REQUEST l
JOIN EMPLOYEE e ON l.Emp_ID = e.Emp_ID
WHERE l.Approval_Status = 'Approved' AND l.Leave_Type = 'Sick';
```

Monitoring Overtime by Department:

SQL

```
SELECT e.Name, d.Dept_Name
FROM EMPLOYEE e
JOIN DEPARTMENT d ON e.Dept_ID = d.Dept_ID
JOIN ATTENDANCE a ON e.Emp_ID = a.Emp_ID
WHERE a.Status = 'Overtime';
```

5. Conclusion

The Workforce Management System successfully establishes a highly organized, normalized relational database capable of handling complex HR operations. By integrating a dynamic Node.js middleware and a Python-based front end, the project bridges the gap between raw database management and user-centric software application. The system effectively guarantees data integrity, prevents logical conflicts, and provides real-time tracking of human resources.

6. Future Scope

- **Automated Payroll Integration:** Linking attendance logs and overtime statuses directly to a payroll calculation module.
- **Biometric Hardware Integration:** Utilizing IoT devices (such as ESP32/RFID scanners) to automatically trigger Clock_In and Clock_Out SQL queries.
- **Advanced Data Visualization:** Expanding the Python front-end to include predictive analytics for employee absenteeism and shift optimization.